

AI-Driven Gate Scheduling for Time-Sensitive Networking: A Generate-and-Verify Approach

Barzan Saeedpour
University of Kurdistan

Sanandaj, Iran
Email: barzansaeedpour@gmail.com

Abstract—Time-Sensitive Networking (TSN) requires transmission schedules that satisfy strict timing, deadline, and non-overlap constraints. Heuristic and learning-based schedulers can generate candidate schedules efficiently, but they do not inherently guarantee correctness. Formal methods, in contrast, can verify feasibility but may become expensive when used as the only scheduling mechanism. This paper presents a generate-and-verify framework for TSN gate scheduling that combines heuristic search, reinforcement learning, formal verification, and lightweight schedule repair. Genetic Algorithm (GA), Simulated Annealing (SA), and Proximal Policy Optimization (PPO) are used as schedule generators, while Z3 is used as a formal verifier and as a baseline exact scheduler. The framework is evaluated on two benchmark scenarios with seven methods: GA, GA with repair, PPO, PPO with repair, SA, SA with repair, and Z3-only. Results show that all methods perform well on the small scenario, while the medium scenario exposes substantial differences in robustness. SA and Z3-only consistently achieve full feasibility, GA remains competitive with occasional failures, and PPO alone fails to produce feasible schedules in the medium case. Repair significantly improves PPO, increasing its success rate from 0.0 to 0.6. These results suggest that hybrid scheduling with formal validation is a practical direction for reliable TSN scheduling.

Index Terms—Time-Sensitive Networking, TSN scheduling, formal verification, Z3, genetic algorithm, simulated annealing, reinforcement learning, PPO, generate-and-verify.

I. INTRODUCTION

Time-Sensitive Networking (TSN) provides deterministic communication capabilities for real-time and safety-critical systems, including industrial automation, embedded platforms, cyber-physical systems, and autonomous systems. In such environments, traffic streams must be transmitted within strict timing bounds while avoiding conflicts on shared network links. Constructing feasible transmission schedules is therefore a central problem in TSN.

The TSN scheduling problem is combinatorial. Even for a moderate number of streams, the number of possible injection-time assignments can grow rapidly. Exact methods can provide strong correctness guarantees, but they may become computationally demanding. Conversely, heuristic and learning-based methods such as Genetic Algorithms (GA), Simulated Annealing (SA), and Reinforcement Learning (RL) can explore large search spaces efficiently, but they may return schedules that violate timing or non-overlap constraints.

This work investigates a hybrid generate-and-verify approach. Candidate schedules are first generated by heuristic or learning-based methods and are then checked using a formal

verifier based on Z3. Invalid schedules can optionally be passed to a conflict-guided repair stage that attempts to reduce deadline violations and link collisions.

The main contributions of this paper are:

- A constrained formulation of TSN gate scheduling with deadline and non-overlap constraints.
- A generate-and-verify scheduling framework combining GA, SA, PPO, and Z3-based formal verification.
- A lightweight repair mechanism for improving invalid schedules.
- An experimental comparison of seven methods across two benchmark scenarios.
- An empirical analysis showing when repair improves feasibility and when formal verification remains essential.

II. BACKGROUND AND MOTIVATION

TSN scheduling can be addressed using exact optimization, metaheuristic search, or learning-based decision making. Exact approaches such as satisfiability modulo theories and constraint optimization are attractive because they can encode timing constraints directly. However, their scalability may become challenging as the number of streams, links, and timing relations increases.

Metaheuristic methods are flexible and often effective for combinatorial optimization. GA performs population-based exploration, while SA performs stochastic local search with controlled acceptance of worse solutions. These methods can generate good schedules quickly, but feasibility is not guaranteed unless constraints are explicitly checked.

Learning-based methods offer another direction by modeling scheduling as sequential decision making. PPO, for example, can learn a policy for selecting injection times. However, RL methods may struggle with sparse feasibility signals and hard constraints, especially when invalid schedules dominate the search space.

The key motivation of this work is that generation and correctness should be separated. A schedule generator may be fast, adaptive, or easy to train, while a verifier provides a common correctness layer. This design allows different generation methods to be evaluated under the same formal feasibility criterion.

III. PROBLEM FORMULATION

We model the TSN network as a directed graph

$$G = (V, E),$$

where V is the set of nodes and E is the set of directed links. Each link $e \in E$ has an associated propagation delay. A set of streams is denoted by

$$\mathcal{S} = \{s_1, s_2, \dots, s_n\}.$$

Each stream s_i is characterized by a transmission duration d_i , a deadline D_i , and a fixed routing path $P_i \subseteq E$.

The decision variable is the injection time x_i of each stream. The objective is to find an assignment of injection times such that all streams meet their deadlines and no two streams overlap on any shared link.

A. Deadline Constraint

For stream s_i , the finish time f_i is computed by propagating its injection time through its path:

$$f_i = x_i + \sum_{e \in P_i} (d_i + \text{delay}(e)) - \text{delay}(e_{\text{last}}).$$

The deadline constraint is:

$$f_i \leq D_i.$$

B. Non-overlap Constraint

If two streams s_i and s_j share a link e , their occupation intervals on that link must not overlap. Let t_i^e and t_j^e denote their start times on link e . With guard band g , the feasibility condition is:

$$t_i^e + d_i + g \leq t_j^e \quad \vee \quad t_j^e + d_j + g \leq t_i^e.$$

C. Penalty Function

For heuristic evaluation, a schedule is assigned a penalty based on deadline lateness and link collisions. A schedule is considered feasible if its total penalty is zero and it is verified by Z3.

IV. PROPOSED FRAMEWORK

The proposed method follows a generate-and-verify pipeline. A candidate schedule is generated by GA, SA, PPO, or Z3-only. The generated schedule is evaluated using the penalty function and then formally checked by Z3. If repair is enabled and the schedule is invalid, the repair stage attempts to improve the schedule and the resulting candidate is re-verified.

Figure 1 illustrates the complete execution flow of the proposed generate-and-verify framework. The pipeline starts from configuration and TSN scenario construction, then selects a scheduling generator among GA, SA, PPO, and Z3-only. Each generated candidate schedule is evaluated using penalty, latency, collision, and deadline-violation metrics. If repair is enabled, conflict-guided adjustments are applied before the final Z3-based verification step. Verified and infeasible schedules are both recorded, and the collected runs are summarized into plots and CSV artifacts.

Algorithm 1 Generate-and-Verify TSN Scheduling

- 1: **Input:** network topology, stream set, scheduling method
- 2: Generate candidate schedule using GA, SA, PPO, or Z3-only
- 3: Compute penalty, latency, and runtime metrics
- 4: Verify schedule feasibility using Z3
- 5: **if** schedule is invalid and repair is enabled **then**
- 6: Identify deadline violations and link collisions
- 7: Modify injection times using conflict-guided repair
- 8: Recompute metrics and re-verify using Z3
- 9: **end if**
- 10: **Return:** schedule, feasibility status, metrics, runtime

A. Schedule Generators

The framework evaluates three representative generator types. GA represents population-based evolutionary search. SA represents stochastic local search. PPO represents reinforcement learning-based sequential decision making. These methods are not combined into a single optimizer; instead, each acts as an independent generator whose output is judged by the same verifier.

B. Formal Verification

Z3 is used to check whether a generated schedule satisfies deadline and non-overlap constraints. In addition, a Z3-only method is included as a baseline. This baseline directly searches for a feasible schedule using formal optimization and provides a reference point for heuristic and learning-based methods.

C. Conflict-Guided Repair

The repair module operates on invalid schedules. It identifies streams that miss deadlines and stream pairs that collide on shared links. Deadline-violating streams are shifted earlier when possible. For collisions, one of the conflicting streams is shifted and the modification with lower penalty is selected. This process is repeated for a bounded number of iterations.

The repair method is lightweight and does not use full UNSAT-core extraction. Nevertheless, it provides a practical mechanism for improving candidate schedules before final verification.

V. EXPERIMENTAL SETUP

The experiments use a directed TSN topology with end systems and switches. The directed links are ES1–SW1, ES2–SW1, SW1–ES3, SW1–ES4, SW1–SW2, and SW2–ES5. Link delays are set to 1 except for SW1–SW2, which has delay 2.

Two scenarios are evaluated. The small scenario contains three streams, while the medium scenario contains five streams with increased contention. All experiments are repeated over 10 random seeds:

11, 22, 33, 44, 55, 66, 77, 88, 99, 111.

The global configuration uses guard band $g = 1$, maximum injection time 20, and 10 independent runs per method. GA

uses a population size of 40 and 30 generations. PPO is trained for 7000 timesteps. SA uses initial temperature 200, cooling rate 0.995, and up to 5000 iterations. Z3-only uses an optimization solver with a timeout of 5000 ms.

A. Evaluation Metrics

The following metrics are reported:

- **Penalty mean:** average penalty over 10 runs.
- **Penalty standard deviation:** variability of penalty.
- **Success rate:** fraction of schedules verified as feasible.
- **Latency mean:** average total latency.
- **Runtime mean:** average generation or solving time.

A successful run is one whose final schedule is formally verified as feasible.

VI. RESULTS

Table I summarizes the results. In the small scenario, most methods achieve perfect feasibility. PPO, PPO+Repair, SA, SA+Repair, and Z3-only obtain zero mean penalty and 100% success rate. GA and GA+Repair each achieve a 0.9 success rate, indicating one failed run out of ten.

The medium scenario provides a clearer distinction between methods. SA, SA+Repair, and Z3-only achieve zero penalty and full success rate. GA remains competitive with a success rate of 0.8. However, GA+Repair performs worse than plain GA in this setting, reducing success rate to 0.6. This suggests that the current repair policy can sometimes disrupt schedules that are already near feasible.

PPO performs poorly in the medium scenario, with mean penalty 124 and no feasible schedules. However, PPO+Repair substantially improves the outcome, reducing the mean penalty to 10 and increasing success rate to 0.6. This indicates that repair is most useful when the initial generator produces invalid but partially recoverable schedules.

VII. FIGURE-BASED ANALYSIS

Figure 2 shows that penalty values are near zero for most methods in the small scenario, while the medium scenario separates the methods more clearly. PPO has the highest penalty in the medium case, whereas SA and Z3-only maintain zero penalty.

Figure 3 confirms that SA, SA+Repair, and Z3-only preserve full feasibility across both scenarios. PPO alone fails in the medium case, while PPO+Repair recovers feasibility in 60% of runs.

Figure 4 shows that PPO inference is very fast, but its runtime advantage does not translate into feasible schedules in the medium scenario. SA and Z3-only remain both reliable and computationally practical for the evaluated scenarios.

VIII. DISCUSSION

The results highlight the importance of formal verification in TSN scheduling. A schedule generator may be fast, but without formal checking it may produce invalid schedules. This is especially visible in the PPO medium scenario, where runtime is low but feasibility is zero.

The results also show that repair is not universally beneficial. PPO benefits substantially from repair because its initial schedules contain many violations but can often be improved. GA, however, does not benefit from the current repair policy and may even degrade. This suggests that repair should be adaptive to the structure and quality of the initial candidate.

Overall, SA and Z3-only provide the strongest performance in the evaluated benchmarks. SA achieves full feasibility with moderate runtime, while Z3-only provides a strong formal baseline. The broader conclusion is that reliable TSN scheduling should combine efficient generation with formal validation rather than relying on heuristic quality alone.

IX. LIMITATIONS

This study has several limitations. First, the topology and traffic scenarios are small to medium in scale, so the results should not be directly generalized to large industrial deployments. Second, the repair mechanism is conflict-guided but does not yet use full UNSAT-core extraction. Third, PPO training is relatively lightweight and may not represent the best possible RL-based scheduler. Finally, the current model does not include richer TSN features such as jitter constraints, queue assignment, dynamic rescheduling, or complete Gate Control List synthesis.

X. CONCLUSION

This paper presented a generate-and-verify framework for TSN gate scheduling. The framework combines GA, SA, PPO, optional repair, and Z3-based formal verification. Experiments on small and medium scenarios show that all methods perform well in the small case, while the medium case exposes major differences in robustness.

SA and Z3-only achieve perfect feasibility across all runs. GA remains competitive but occasionally fails. PPO alone is unreliable in the medium scenario, but repair significantly improves its performance. These findings support the central claim that hybrid scheduling with formal verification is a practical and reliable direction for TSN scheduling.

Future work should extend the framework to larger topologies, stronger verifier-guided repair, UNSAT-core based conflict extraction, dynamic rescheduling, and more realistic TSN gate control models.

REPRODUCIBILITY NOTES

The implementation uses Python with Z3, NetworkX, NumPy, Matplotlib, Gymnasium, Stable-Baselines3, and DEAP. The pipeline generates CSV result files and comparison plots, including penalty, success rate, and runtime figures. The LaTeX source automatically loads generated figures when they are placed in the project root or in the configured figure path.

The source code and experimental artifacts are available at:

GitHub Repository

TABLE I
SUMMARY OF EXPERIMENTAL RESULTS

Scenario	Algorithm	Penalty Mean	Penalty Std	Success Rate	Latency Mean	Runtime Mean (s)
Small	GA	1.0	3.0	0.9	15.0	0.084555
Small	GA+Repair	1.0	3.0	0.9	15.0	0.083451
Small	PPO	0.0	0.0	1.0	15.0	0.003680
Small	PPO+Repair	0.0	0.0	1.0	15.0	0.003477
Small	SA	0.0	0.0	1.0	15.0	0.006112
Small	SA+Repair	0.0	0.0	1.0	15.0	0.006620
Small	Z3-only	0.0	0.0	1.0	15.0	0.004474
Medium	GA	2.0	4.0	0.8	30.0	0.117372
Medium	GA+Repair	6.0	8.0	0.6	30.0	0.117123
Medium	PPO	124.0	28.0	0.0	30.0	0.005341
Medium	PPO+Repair	10.0	12.6491	0.6	30.0	0.005882
Medium	SA	0.0	0.0	1.0	30.0	0.015126
Medium	SA+Repair	0.0	0.0	1.0	30.0	0.028914
Medium	Z3-only	0.0	0.0	1.0	30.0	0.010286

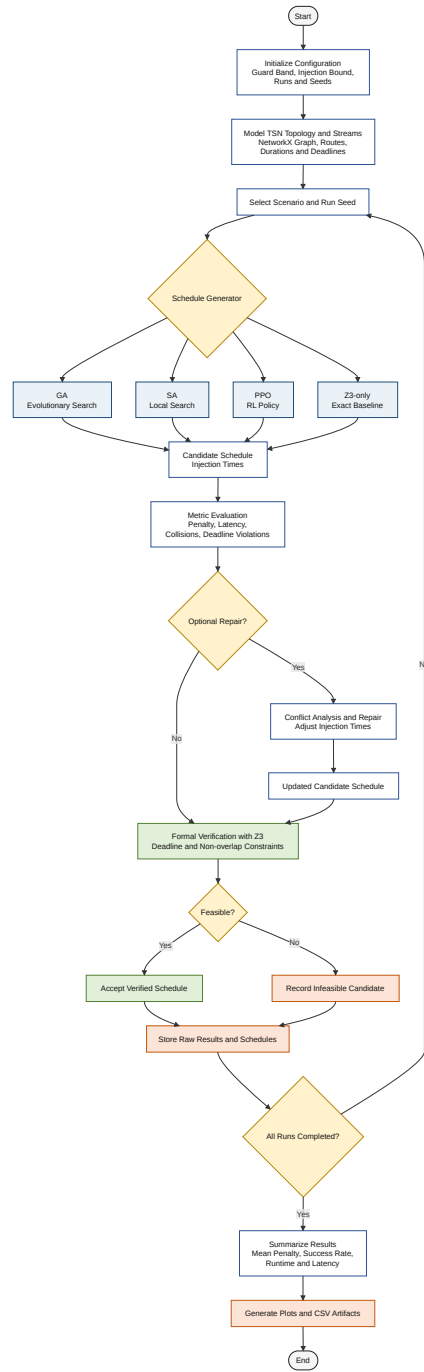


Fig. 1. Flowchart of the proposed generate-and-verify TSN scheduling framework.

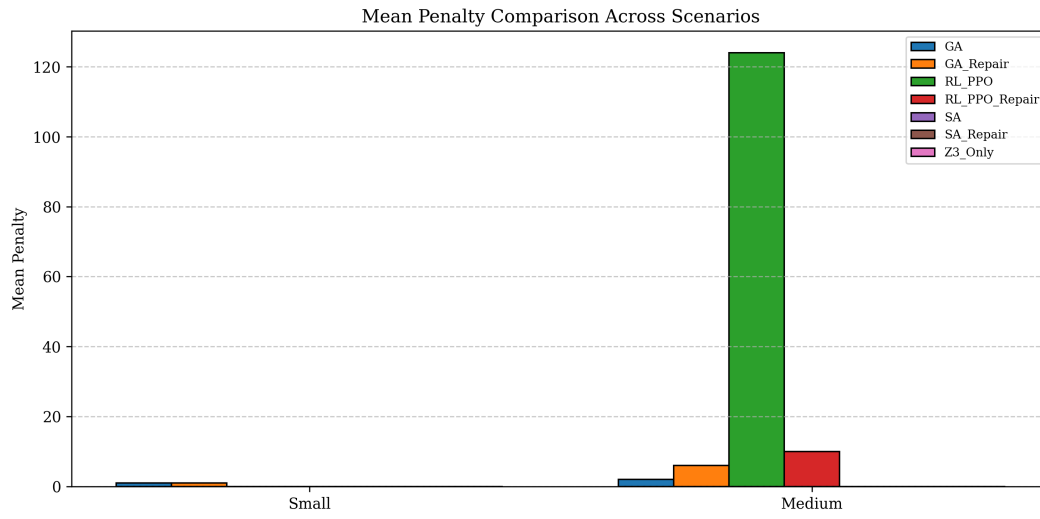


Fig. 2. Mean penalty comparison across scenarios.

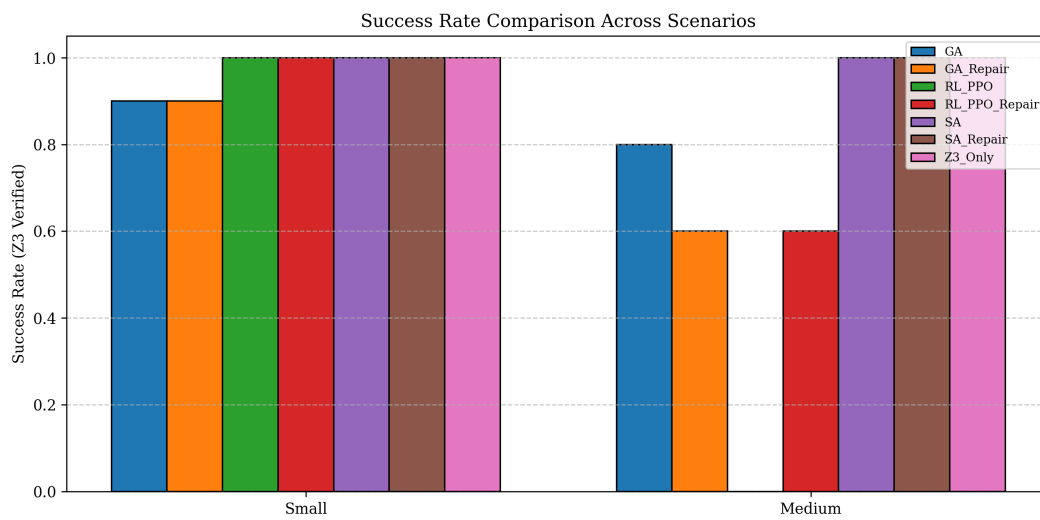


Fig. 3. Success rate comparison across scenarios.

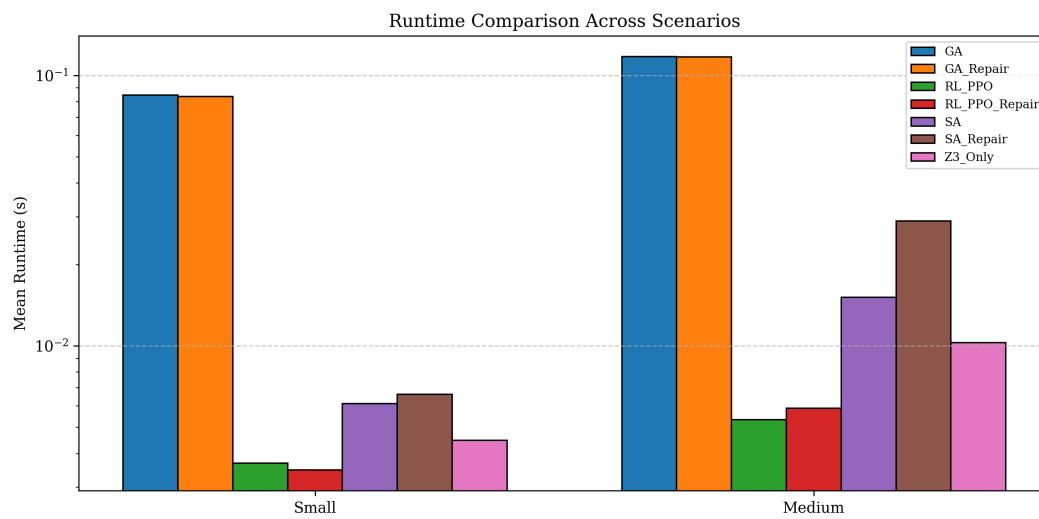


Fig. 4. Runtime comparison across scenarios.